

Machine Problem 6: Primitive Disk Device Driver

Nithin Gowtham Saravanan

UIN: 337001954

CSCE611: Operating Systems

Contents

Assigned Tasks	2
System Design	2
Disk Device Driver Architecture	3
Method of Implementation	4
1. Identify Busy Waiting	4
2. NonBlockingDisk Override	5
3. Thread-Safe Disk (Bonus Option 2)	5
4. Interrupt Hint (Lightweight Use of IRQ14)	6
CODE Description	6
thread.H/thread.C/Scheduler.H/Scheduler.C	7
nonblocking_disk.H	7
nonblocking_disk.C	9
macros_config.H	13
Kernel.C	14
Bonus Option 1 - Design of a thread-safe disk system	15
Bonus Option 2: Implementation of Thread-Safe Disk	16
Bonus Option 3: Using Interrupts for Concurrency	16
TESTING	17
A. Baseline Implementation – MP6 requirements for non-blocking disk	17
B. NonBlockingDisk with Thread-Safe Lock (Bonus 1 & 2)	18
C. NonBlockingDisk with Thread-Safe Lock + Disk Interrupt (Bonus 3)	19

Assigned Tasks

Main: Completed.

Bonus Option 1: Design of a thread-safe disk system – Completed.

Bonus Option 2: Implementation of a thread-safe disk system – Completed.

Bonus Option 3: Using Interrupts for Concurrency - Completed.

Bonus Option 4: Not applicable.

System Design

The goal of MP6 is to preserve the **blocking** semantics of `SimpleDisk::read()` and `write()` while avoiding **busy waiting** inside the kernel. The solution is to introduce a `NonBlockingDisk` that:

1. **Reuses** all ATA protocol logic from `SimpleDisk`.
2. **Overrides** only `wait_while_busy()` to avoid tight polling when the disk is busy.
3. Optionally adds:
 - A **per-disk lock** for thread safety.
 - An **interrupt hint** from IRQ14 to help the waiting thread detect disk progress.

High-level components:

- **SimpleDisk**
 - Low-level disk implementation that understands the ATA controller and LBA28 protocol.
 - Still responsible for issuing commands, polling status registers, and transferring sectors.
- **NonBlockingDisk**
 - Subclass of `SimpleDisk`.
 - Overrides `wait_while_busy()` to cooperate with the scheduler instead of spinning.
 - Optionally adds:
 - A simple **spin-with-yield lock** (`lock_disk()` / `unlock_disk()`) to serialize disk accesses.
 - A small **IRQ14 handler** that sets `irq_fired` when the disk interrupts, allowing `wait_while_busy()` to react quickly.
- **Scheduler / Threads**
 - The scheduler from MP5 is reused as a FIFO scheduler (no RR required for this MP).
 - `System::SCHEDULER` is used to **yield the CPU** when the disk is busy.

- Threads call `System::DISK->read()` and `write()`; they block in the driver, but the **CPU remains available** for other threads.
- **System Integration**
 - `System::DISK` is changed from `SimpleDisk` to `NonBlockingDisk`.
 - The same kernel build can run:
 - With or without `_USES_SCHEDULER_`.
 - With or without `_THREADSAFE_DISK_`.
 - With or without `_DISK_MASK_INTERRUPTS_`.

Disk Device Driver Architecture

1. SimpleDisk as the hardware-facing layer

- `SimpleDisk` knows the ATA registers, issues READ/WRITE commands (`ide_ata_issue_command`), polls disk status (`ide_polling / get_status`), and transfers 512B sectors via PIO on port 0x1F0.
- It exposes `read(block, buf)` and `write(block, buf)` as a simple blocking API.

2. NonBlockingDisk as a thin wrapper over SimpleDisk

- `NonBlockingDisk` **inherits** from `SimpleDisk` and overrides only the *waiting policy* (`wait_while_busy()`), leaving all ATA protocol and data transfer logic in `SimpleDisk` untouched.
- So every read / write still goes through the same low-level code, but the “wait while drive is busy” behavior is improved.

3. Scheduler-aware busy-wait removal

- When `_USES_SCHEDULER_` is enabled, `NonBlockingDisk::wait_while_busy()` replaces the tight while (`is_busy()`) {} loop with:
 - while (`is_busy()`) { `System::SCHEDULER->yield()`; } (plus safety checks),
- This means the *calling thread still blocks logically*, but the CPU is free to run other ready threads instead of spinning in the driver.

4. Thread safety

- With `_THREADSAFE_DISK_` defined, `NonBlockingDisk` adds a simple spin-with-yield lock (`lock_disk()/unlock_disk()` guarded by `disable_interrupts/enable_interrupts`).
- Its read/write override becomes: acquire lock → call `SimpleDisk::read/write` → release lock.
- This guarantees only one thread talks to the ATA controller at a time, so PIO transfers don’t get interleaved.

5. System-wide access through the System singleton

- The kernel sets `System::DISK = new NonBlockingDisk(System::DISK_SIZE);`.

- All code (e.g., fun2, fun4) uses `System::DISK->read/write`, so switching from `SimpleDisk` to `NonBlockingDisk` required no call-site changes—only the construction changed.

6. Threads + scheduler driving disk I/O cooperatively

- Worker threads (fun2, and optionally fun4 under `_THREADSAFE_DISK_`) repeatedly call `System::DISK->read/write` inside loops.
- Whenever the disk is busy, `wait_while_busy()` yields; the Scheduler picks another thread from the ready queue (fun1, fun3, or another disk thread).
- Result: disk calls are still synchronous from the thread's perspective, but the **CPU is always doing useful work** while the disk is busy.

7. Interrupt hint integration (not full bottom half)

- If `_DISK_MASK_INTERRUPTS_` is enabled, `NonBlockingDisk` also registers as IRQ14 handler and sets an `irq_fired` flag on IDE interrupts.
- `wait_while_busy()` checks this flag to re-check disk status promptly, improving responsiveness without changing the fundamental synchronous design.

Method of Implementation

1. Identify Busy Waiting

In `SimpleDisk`, busy waiting occurs here:

```
bool SimpleDisk::is_busy() {
    return (get_status() & ATA_STATUS_BSY);
}

void SimpleDisk::wait_while_busy() {
    while (is_busy()) { /* busy loop */ }
}
```

This function is invoked from both:

- `SimpleDisk::ide_polling()` – after issuing a command, waits for BSY to clear.
- `SimpleDisk::ide_ata_issue_command()` – before issuing a new command, waits for the drive to be idle.

This loop **ties up the CPU**, even with a scheduler present.

2. NonBlockingDisk Override

NonBlockingDisk inherits from SimpleDisk and overrides wait_while_busy():

- When `_USES_SCHEDULER_` is **not** defined, it simply calls `SimpleDisk::wait_while_busy()` (original busy loop) to keep the non-scheduler baseline working.
- When `_USES_SCHEDULER_` is defined, it continuously:
 - Checks `is_busy()`.
 - If busy: calls `System::SCHEDULER->yield()` to give the CPU to other threads.
 - If not busy: returns, letting the original ATA logic proceed.

With `_DISK_MASK_INTERRUPTS_` also defined, the override:

- Watches a volatile flag `irq_fired` that is set by the IRQ14 handler.
- On each loop:
 - If `irq_fired` is set: clear it and re-check `is_busy()` immediately.
 - Otherwise: `yield()` to give other threads a chance to run.

This respects the original ATA protocol (SimpleDisk still handles all status/error bits) but removes tight CPU spinning.

3. Thread-Safe Disk (Bonus Option 2)

To support **multiple threads** calling `read()/write()` concurrently:

- A **per-disk lock** is added as a simple spin-lock with yields, to serialize active disk operations.
- Lock is implemented using:
 - `volatile bool lock_held;`
 - `Machine::disable_interrupts() / Machine::enable_interrupts()` around changes to `lock_held` (to avoid lost updates on a uniprocessor).
 - Optional `System::SCHEDULER->yield()` in the spin loop so the lock holder can run.

The wrapper methods:

```
void NonBlockingDisk::read(unsigned long block, unsigned char* buf) {
    lock_disk();
    SimpleDisk::read(block, buf);
    unlock_disk();
}
```

```
void NonBlockingDisk::write(unsigned long block, unsigned char* buf) {
    lock_disk();
    SimpleDisk::write(block, buf);
}
```

```
unlock_disk();
}
```

This guarantees:

- At most one thread drives the ATA controller at a time.
- Blocking still happens via the overridden `wait_while_busy()`, which now yields while the disk is busy.
- Concurrent callers are queued by the lock, not by the disk hardware.

4. Interrupt Hint (Lightweight Use of IRQ14)

To reduce pure polling further:

- `NonBlockingDisk` also inherits from `InterruptHandler` when `_USES_SCHEDULER_` is defined.
- The constructor registers itself as the handler for IRQ14 (primary IDE):

```
InterruptHandler::register_handler(14, this);
```

- `handle_interrupt()` simply sets `irq_fired = true` when `_DISK_MASK_INTERRUPTS_` is defined; otherwise it does nothing (only silences the default handler warning).

This design:

- Does **not** move data transfers to a bottom half.
- Does **not** maintain a disk request queue.
- Keeps all logic in the synchronous top half; interrupts are only a hint to wake the waiting thread faster.

Thus the Option 3 requirements (full top/bottom half with queued requests) are **not** fully implemented.

CODE Description

This implementation modifies the following files:

- `thread.H`
- `thread.C`
- `scheduler.H`
- `scheduler.C`
- `nonblocking_disk.H`
- `nonblocking_disk.C`
- `macros_config.H`
- `kernel.C`

thread.H/thread.C/Scheduler.H/Scheduler.C

- All these files are ported from MP5. Please refer to the documentation of MP5 for a detailed explanation of the implementation procedures.
- In this MP6, I have only enabled the FIFO scheduling for project usage and its first validated with the simple disk implementation to ensure that the scheduler works perfectly.
- On top of this, the upcoming changes are made.

nonblocking_disk.H

Class Declaration

- **NonBlockingDisk : public SimpleDisk (+ InterruptHandler if _USES_SCHEDULER_)**
Extends SimpleDisk to replace busy waiting with scheduler-aware waiting, and optionally handles IRQ14.

Public Interface

- **NonBlockingDisk(unsigned int _size)**
Creates the disk and registers IRQ14 when the scheduler is enabled.
- **wait_while_busy()**
Core improvement: replaces tight busy loop with yielding. With _DISK_MASK_INTERRUPTS_, detects IRQ14 to re-check disk status sooner.

Thread-safe Disk

- **read(block, buf)**
Acquires per-disk lock, performs the read, releases lock.
- **write(block, buf)**
Same as above, but for writes. Ensures only one thread interacts with disk hardware at a time.

Interrupt Handling

- **handle_interrupt(REGS*)**
Silences default IRQ14 message; with _DISK_MASK_INTERRUPTS_, sets a flag for wait_while_busy() to react to.

Private Members

Per-Disk Lock

- **volatile bool lock_held** — tracks whether the lock is taken.
- **lock_disk()** — spin-with-yield lock acquisition using interrupt masking for safety.

- **unlock_disk()** — releases the lock safely.

Interrupt Hint

- **volatile bool irq_fired** — set on IRQ14; used by wait_while_busy() to re-check disk immediately.

```
class NonBlockingDisk
#ifdef _USES_SCHEDULER_
    : public SimpleDisk, public InterruptHandler
#else
    : public SimpleDisk
#endif
{
public:
    explicit NonBlockingDisk(unsigned int _size);

    // Core non-blocking behavior: override SimpleDisk's busy wait.
    virtual void wait_while_busy() override;

#ifdef _THREADSAFE_DISK_
    // Thread-safe wrappers around SimpleDisk's read/write.
    virtual void read(unsigned long _block_no, unsigned char* _buf) override;
    virtual void write(unsigned long _block_no, unsigned char* _buf) override;
#endif

#ifdef _USES_SCHEDULER_
    // IRQ14 handler.
    // - Options 1 & 2: just silences the default handler.
    // - Option 3 (_DISK_MASK_INTERRUPTS_): sets a flag observed by
    //   wait_while_busy().
    virtual void handle_interrupt(REGS* regs) override;
#endif

private:
#ifdef _THREADSAFE_DISK_
    volatile bool lock_held;
    void lock_disk();
    void unlock_disk();
#endif

#if defined(_USES_SCHEDULER_) && defined(_DISK_MASK_INTERRUPTS_)
    // Set by IRQ14 to tell the waiting thread that the disk changed state.
    volatile bool irq_fired;
#endif
};
```


nonblocking_disk.C

NonBlockingDisk::NonBlockingDisk(unsigned int _size)

- Calls SimpleDisk(_size) to reuse all low-level IDE/ATA logic and block layout.
- Initializes optional features: lock_held for thread-safe access and irq_fired for interrupt-hinted waiting.
- When _USES_SCHEDULER_ is enabled, registers itself as the handler for IRQ14 so IDE disk interrupts are delivered to this object.

```

/*-----
/* CONSTRUCTOR */
/*-----

NonBlockingDisk::NonBlockingDisk(unsigned int _size)
    : SimpleDisk(_size)
#ifdef _THREADSAFE_DISK_
    , lock_held(false)
#endif
#if defined(_USES_SCHEDULER_) && defined(_DISK_MASK_INTERRUPTS_)
    , irq_fired(false)
#endif
{
#ifdef _USES_SCHEDULER_
    /* For Options 1 & 2 we don't *use* the interrupt, we just silence
       the default "NO DEFAULT INTERRUPT HANDLER" messages.
       For Option 3, we also use it to set irq_fired.*/
    InterruptHandler::register_handler(14, this);
#endif
}

```

void NonBlockingDisk::wait_while_busy()

Case 1: _USES_SCHEDULER_ not defined

- Simply delegates to SimpleDisk::wait_while_busy(), preserving original busy-loop behavior.
- Ensures correctness even when the OS has no scheduler support.

```

void NonBlockingDisk::wait_while_busy()
{
#ifdef _USES_SCHEDULER_

    // No scheduler: fall back to original busy-wait from SimpleDisk.
    SimpleDisk::wait_while_busy();
    return;

#else // _USES_SCHEDULER_ is defined

```

Case 2: _USES_SCHEDULER_ defined, _DISK_MASK_INTERRUPTS_ not defined

- Implements scheduler-based polling: while `is_busy()` is true, calls `System::SCHEDULER->yield()` instead of spinning.
- Reduces CPU waste by letting other threads run while the disk remains busy, but still only uses status polling (no interrupts).

```
#else // _USES_SCHEDULER_ is defined

# ifdef _DISK_MASK_INTERRUPTS_

    /* - Loop while is_busy() is true.
       - Between checks, yield the CPU.
       - If IRQ14 fires, handle_interrupt() sets irq_fired; here we
         notice it, clear it, and immediately re-check is_busy(). */

    assert(System::SCHEDULER != nullptr);

    while (is_busy()) {
        // If an interrupt fired, the disk likely changed state; just clear
        // the flag and let the top of the loop re-check is_busy().
        if (irq_fired) {
            irq_fired = false;
            continue;
        }

        // Let other threads run while we wait for BSY to drop.
        System::SCHEDULER->yield();
    }

    return;
```

Case 3: _USES_SCHEDULER_ + _DISK_MASK_INTERRUPTS_ defined

- Waits in a loop on `is_busy()`, but uses an interrupt “hint”: when `irq_fired` is set by IRQ14, it clears the flag and rechecks immediately.
- Between checks, yields through `System::SCHEDULER->yield()`, combining cooperative scheduling with interrupt-driven hints.
- Keeps the ATA protocol intact because the final status checks and error handling still happen inside `SimpleDisk::ide_polling()`.

```

# else // !_DISK_MASK_INTERRUPTS_

    /* Options 1 & 2: scheduler-based polling. */
    while (is_busy()) {
        if (System::SCHEDULER != nullptr) {
            System::SCHEDULER->yield();
        } else {
            // Safety fallback if scheduler isn't initialized for some reason.
            SimpleDisk::wait_while_busy();
            break;
        }
    }
    return;

# endif // _DISK_MASK_INTERRUPTS_

```

void NonBlockingDisk::lock_disk() (only with *_THREADSAFE_DISK_*)

- Implements a simple per-disk spin-lock protected by disabling interrupts while testing and setting lock_held.
- If the lock is already taken, re-enables interrupts and, when a scheduler is present, yields the CPU so the lock holder can make progress.
- Ensures only one thread at a time executes SimpleDisk::read/write, avoiding concurrent access to the shared IDE controller registers.

```

void NonBlockingDisk::lock_disk()
{
    // Simple spin-lock with scheduler-friendly yielding when available.
#ifdef _USES_SCHEDULER_
    assert(System::SCHEDULER != nullptr);
#endif

    for (;;) {
        Machine::disable_interrupts();
        if (!lock_held) {
            lock_held = true;
            Machine::enable_interrupts();
            return;
        }
        Machine::enable_interrupts();
    }

#ifdef _USES_SCHEDULER_
    // Let the lock holder (and other threads) run.
    if (System::SCHEDULER != nullptr) {
        System::SCHEDULER->yield();
    }
#endif

    // If no scheduler, this is just a busy loop.
}

```

void NonBlockingDisk::unlock_disk() (only with `_THREADSAFE_DISK_`)

- Disables interrupts, clears `lock_held`, then re-enables interrupts to avoid races with other threads attempting to acquire the lock.
- Forms the paired critical section exit for `lock_disk()`, completing the mutual exclusion protocol around disk operations.

```
void NonBlockingDisk::unlock_disk()
{
    Machine::disable_interrupts();
    lock_held = false;
    Machine::enable_interrupts();
}
```

void NonBlockingDisk::read(unsigned long _block_no, unsigned char* _buf)*(only with `_THREADSAFE_DISK_`)*

- Acquires the per-disk lock via `lock_disk()` before touching the hardware.
- Delegates the actual I/O to `SimpleDisk::read`, preserving existing ATA/PIO behavior.
- Releases the lock with `unlock_disk()` so other threads can perform disk operations.

```
void NonBlockingDisk::read(unsigned long _block_no, unsigned char* _buf)
{
    lock_disk();
    SimpleDisk::read(_block_no, _buf);
    unlock_disk();
}
```

void NonBlockingDisk::write(unsigned long _block_no, unsigned char* _buf)*(only with `_THREADSAFE_DISK_`)*

- Mirrors `read()`: acquires the disk lock, calls `SimpleDisk::write`, then releases the lock.
- Guarantees that even concurrent writers (and readers) use the disk in a serialized, race-free manner.

```
void NonBlockingDisk::write(unsigned long _block_no, unsigned char* _buf)
{
    lock_disk();
    SimpleDisk::write(_block_no, _buf);
    unlock_disk();
}
```

void NonBlockingDisk::handle_interrupt(REGS* /*regs*/)

(only with `_USES_SCHEDULER_`)

With `_DISK_MASK_INTERRUPTS_` defined

- Called on IRQ14 whenever the IDE controller signals an event.
- Simply sets `irq_fired = true`, acting as a lightweight bottom-half trigger without touching scheduler or interrupt state.
- `wait_while_busy()` observes this flag and reacts by rechecking `is_busy()` promptly, improving responsiveness.

Without `_DISK_MASK_INTERRUPTS_`

- Implements an empty handler (`void`)0; to suppress the default “NO DEFAULT INTERRUPT HANDLER” warning.
- Leaves disk behavior purely polling-based while still keeping the interrupt subsystem quiet

```
#ifdef _USES_SCHEDULER_
void NonBlockingDisk::handle_interrupt(REGS* /*regs*/)
{
#ifdef _DISK_MASK_INTERRUPTS_
    /* record the fact that an IDE interrupt occurred. The waiting
       thread in wait_while_busy() will see this and re-check the disk. */
    irq_fired = true;
#else
    // Options 1 & 2 - don't do anything
    (void)0;
#endif
}
}
```

macros_config.H

- defines global flags used throughout the project to ensure backwards compatibility when turning on/off specific scenarios. These will be widely used in the bonus options

```
#define _USES_SCHEDULER_ // uncomment to enable FIFO scheduling
#define _THREADSAFE_DISK_ // uncomment to verify thread safe disk behavior (MP6 Option 2)
#define _DISK_MASK_INTERRUPTS_ // Enable interrupt-driven disk bottom half (MP6 Option 3)
// #define _TERMINATING_FUNCTIONS_ // uncomment to terminate threads 1 and 2 in kernel.c
// #define _RR_SCHEDULER_ // uncomment to enable Round-Robin
```

Kernel.C

1. Disk Construction – Uncomment the disk construction for nonblocking

```
// System::DISK = new SimpleDisk(System::DISK_SIZE); // Replace this with commented code below when you are ready!

// #define _USES_SCHEDULER_
// // The NonBlockingDisk uses a scheduler.
System::DISK = new NonBlockingDisk(System::DISK_SIZE);
```

2. Thread Functions – To test thread safe mechanisms

- fun4():
 - When `_THREADSAFE_DISK_` is **not** defined:
 - Remains a purely CPU-bound ticker.
 - When `_THREADSAFE_DISK_` **is** defined:
 - Becomes a second disk worker:
 - Uses different starting read_block and write_block indices.
 - Reads, prints, writes with a distinct pattern (j + 100) % 256.
 - Yields back to thread1.
 - This stresses the per-disk lock and ensures thread-safe access.

```
/* -----
Thread-safe disk test behavior (when _THREADSAFE_DISK_ is defined):
Make FUN 4 also hammer the disk concurrently with FUN 2.
This stresses the per-disk lock and non-blocking wait_while_busy().
----- */

unsigned char buf[DISK_BLOCK_SIZE];
int read_block = 5; // start at a different block than FUN 2 (1)
int write_block = 9; // arbitrary - you can overlap on purpose if you want

Console::puts("FUN 4 DISK TEST MODE ENABLED\n");

for (int j = 0;; j++) {

    Console::puts("FUN 4 IN DISK ITERATION["); Console::puti(j); Console::puts("]\n");

    /* -- Read */
    Console::puts("FUN 4: Reading Block "); Console::puti(read_block); Console::puts(" from disk...\n");
    System::DISK->read(read_block, buf);

    Console::puts("\nFUN 4: Content of block is:");
    for (int i = 0; i < DISK_BLOCK_SIZE; i++) {
        Console::putui((unsigned int)buf[i]);
        /* Write a different pattern than FUN 2 so we can see who wrote what. */
        buf[i] = (j + 100) % 256; // distinguish from FUN 2's j % 256
    }
    Console::puts("\n");

    /* -- Write updated contents back */
    Console::puts("FUN 4: Writing buffer to Block "); Console::puti(write_block); Console::puts(" on disk...\n");
    System::DISK->write(write_block, buf);
    Console::puts("\nFUN 4: Done writing\n");

    /* -- Advance blocks (wrap within 0..9 like FUN 2) */
    write_block = read_block;
    read_block = (read_block + 1) % 10;

    /* -- Give up the CPU */
    pass_on_CPU(thread1);
}
```

- Thread stacks were increased (e.g., from 1024 bytes to 4096 bytes) to provide safer space for deep call stacks (ATA code + scheduler + console output).

```

/* -- LET'S CREATE SOME THREADS... */

Console::puts("CREATING THREAD 1...\n");
char* stack1 = new char[4096];
thread1 = new Thread(fun1, stack1, 4096);
Console::puts("DONE\n");

Console::puts("CREATING THREAD 2...");
char* stack2 = new char[4096];
thread2 = new Thread(fun2, stack2, 4096);
Console::puts("DONE\n");

Console::puts("CREATING THREAD 3...");
char* stack3 = new char[4096];
thread3 = new Thread(fun3, stack3, 4096);
Console::puts("DONE\n");

Console::puts("CREATING THREAD 4...");
char* stack4 = new char[4096];
thread4 = new Thread(fun4, stack4, 4096);
Console::puts("DONE\n");

```

Bonus Option 1 - Design of a thread-safe disk system

Key race conditions if multiple threads access SimpleDisk concurrently:

- Interleaved ATA commands:
 - One thread might issue a READ, another immediately issues a WRITE before the first request's data has been fully transferred.
 - Status and error bits in the controller would no longer correspond to a single active request.
- Mixed PIO data transfers:
 - If two threads interleave reads on port 0x1F0, they will mix sector words, corrupting both blocks.
- Lost wakeups / inconsistent polling:
 - One thread might wait in `ide_polling()` while another issues a new command, confusing the busy/ready state.

Proposed solution:

- Serialize all access to the hardware with a **per-disk lock**.
- Maintain blocking semantics at the **request level**:
 - A thread “holds the disk” from issuing the ATA command until its transfer is complete.
- Keep the design simple:
 - No per-request queues.
 - No reordering or overlapping I/O; just safe sequential access.

This design is implemented by the `_THREADSAFE_DISK_` path in `NonBlockingDisk`.

Bonus Option 2: Implementation of Thread-Safe Disk

Implemented features:

- volatile bool `lock_held` within `NonBlockingDisk`.
- `lock_disk()` and `unlock_disk()` protected by `Machine::disable_interrupts()` / `enable_interrupts()` to avoid race conditions.
- Scheduler-friendly spin - Waiting threads yield the CPU instead of busy looping.

Behavior:

- If `fun2()` and `fun4()` both attempt disk I/O:
 - One thread acquires the lock, performs its full read+write, then releases the lock.
 - The other thread waits (with yields), then runs its own operation.
- `wait_while_busy()` still ensures non-busy waiting during hardware busy periods.

Note: Implementations are already discussed in the code descriptions and the output is described in the testing section

Bonus Option 3: Using Interrupts for Concurrency

Not fully implemented.

- Implemented only a minimal interrupt handler:
 - IRQ14 is registered and `handle_interrupt()` sets `irq_fired = true` when `_DISK_MASK_INTERRUPTS_` is defined.
 - `wait_while_busy()` uses this flag as a **hint** to re-check status promptly.
- There is no bottom-half processing, disk request queue, or completion-based wakeup.
- Therefore, this does not satisfy the full Option 3 requirements as described in the MP handout.

TESTING

A. Baseline Implementation – MP6 requirements for non-blocking disk

Configuration

- `_USES_SCHEDULER_ defined`
- `_THREADSAFE_DISK_ and _DISK_MASK_INTERRUPTS_ undefined`
- `System::DISK = new NonBlockingDisk(System::DISK_SIZE);`

Procedure

1. Enable `_USES_SCHEDULER_` in `macros_config.H`.
2. Ensure `_THREADSAFE_DISK_` and `_DISK_MASK_INTERRUPTS_` are disabled.
3. Build and run: **`make run > kernel_log_baseline_nonblocking.txt 2>&1`**
4. Observe the sequence of thread outputs:
 - fun1 prints its burst of ticks.
 - fun2 prints “Reading Block ... / Content of block ... / Writing buffer ...”.
 - fun3 prints its burst of ticks.
 - fun4 behaves as the CPU-bound ticker (no disk I/O in this configuration).

```

Document List
Name      Ext.
new 1
kernel_log_baseline_... .txt
kernel_log_threadsafe_... .txt
kernel_log_disk_mask_... .txt

100 FUN 4: TICK [6]
101 FUN 4: TICK [7]
102 FUN 4: TICK [8]
103 FUN 4: TICK [9]
104 FUN 1: IN ITERATION[2]
105 FUN 1: TICK [0]
106 FUN 1: TICK [1]
107 FUN 1: TICK [2]
108 FUN 1: TICK [3]
109 FUN 1: TICK [4]
110 FUN 1: TICK [5]
111 FUN 1: TICK [6]
112 FUN 1: TICK [7]
113 FUN 1: TICK [8]
114 FUN 1: TICK [9]
115 FUN 2: IN ITERATION[1]
116 FUN 2: Reading Block 2 from disk...
117 FUN 3: IN BURST[2]
118 FUN 3: TICK [0]
119 FUN 3: TICK [1]
120 FUN 3: TICK [2]
121 FUN 3: TICK [3]
122 FUN 3: TICK [4]
123 FUN 3: TICK [5]
124 FUN 3: TICK [6]
125 FUN 3: TICK [7]
126 FUN 3: TICK [8]
127 FUN 3: TICK [9]
128 FUN 4: IN BURST[2]
129 FUN 4: TICK [0]
130 FUN 4: TICK [1]
131 FUN 4: TICK [2]
132 FUN 4: TICK [3]
133 FUN 4: TICK [4]
134 FUN 4: TICK [5]
135 FUN 4: TICK [6]
136 FUN 4: TICK [7]
137 FUN 4: TICK [8]
138 FUN 4: TICK [9]
139 FUN 1: IN ITERATION[3]
140 FUN 1: TICK [0]
141 FUN 1: TICK [1]
142 FUN 1: TICK [2]
143 FUN 1: TICK [3]
144 FUN 1: TICK [4]
145 FUN 1: TICK [5]
146 FUN 1: TICK [6]
147 FUN 1: TICK [7]
148 FUN 1: TICK [8]
149 FUN 1: TICK [9]
150
151 FUN 2: Content of block 1 is:***
152 FUN 2: Writing buffer to block 1 on disk...
153 FUN 3: IN BURST[3]
154 FUN 3: TICK [0]
155 FUN 3: TICK [1]
156 FUN 3: TICK [2]
157 FUN 3: TICK [3]

```

Expected Behavior

- While fun2 is doing disk I/O, the system **does not** freeze in a busy loop:
 - The scheduler continues to run fun1 and fun3 in between disk checks.

- The output shows interleaving of fun2's disk messages with fun1/fun3 ticks, proving that the CPU is not monopolized by disk waits.
- Disk contents remain consistent across iterations (block patterns change in a controlled fashion: $j \% 256$).

Note: A detailed log is stored at kernel_log_baseline_nonblocking.txt. This is executed with the command make run > kernel_log_baseline_nonblocking.txt 2>&1

B. NonBlockingDisk with Thread-Safe Lock (Bonus 1 & 2)

Configuration

- `_USES_SCHEDULER_` and `_THREADSAFE_DISK_` **defined**
- `_DISK_MASK_INTERRUPTS_` **undefined**
- `System::DISK = new NonBlockingDisk(System::DISK_SIZE);`

Procedure

1. Enable `_USES_SCHEDULER_` and `_THREADSAFE_DISK_` in `macros_config.H`.
2. Ensure `_DISK_MASK_INTERRUPTS_` is disabled.
3. Build and run: **`make run > kernel_log_threadsafe.txt 2>&1`**
4. Observe:
 - `fun2` and `fun4` both announce disk I/O (“Reading Block ... / Content of block is ... / Writing buffer ...”).
 - Blocks used by `fun2` and `fun4` overlap or move through a shared range (0–9), causing alternating patterns in the same blocks.

[illegible]

Expected Behavior

- **No corruption** of the ATA protocol:
 - For any given read/write sequence, you see a clean 512-byte pattern (e.g. all <21>, then all <22>, etc.), not mixed values from two threads.
- The log shows interleaving between fun2 and fun4 at the **call level**, but not within a single sector transfer.
- This demonstrates:
 - The per-disk lock properly serializes actual hardware accesses.
 - wait_while_busy() still yields while the disk is busy, avoiding CPU spin.

Note: A detailed log is stored at kernel_log_threadsafe.txt. This is executed with the command `make run > kernel_log_threadsafe.txt 2>&1`

C. NonBlockingDisk with Thread-Safe Lock + Disk Interrupt (Bonus 3)**Configuration**

- `_USES_SCHEDULER_`, `_THREADSAFE_DISK_` and `_DISK_MASK_INTERRUPTS_` are **defined**
- `System::DISK = new NonBlockingDisk(System::DISK_SIZE);`

Procedure

1. Enable `_USES_SCHEDULER_`, `_THREADSAFE_DISK_`, and `_DISK_MASK_INTERRUPTS_` in `macros_config.H`.
2. Build and run: `make run > kernel_log_disk_mask_interrupts.txt 2>&1`
3. Observe similar output to Scenario B, but now:
 - There is also a line at boot indicating that an interrupt handler is installed on IRQ14.
 - Disk I/O still behaves correctly, with no corruption.

Expected Behavior

- From a functional point of view, behavior matches Scenario B:
 - Blocking reads/writes still work.
 - Multiple threads can safely use the disk.
- Internally:
 - IRQ14 triggers `NonBlockingDisk::handle_interrupt()`, setting `irq_fired`.
 - `wait_while_busy()` notices `irq_fired` and re-checks status promptly, reducing the number of pure polling loops.
- Full Option 3 is not claimed; no bottom-half or queued requests are present.

Note: A detailed log is stored at kernel_log_disk_mask_interrupts.txt. This is executed with the command make run > kernel_log_disk_mask_interrupts.txt 2>&1